LLM-Grounded Cross-Domain Bayesian Inference:
From Word Priors to Persistent Belief Networks

Anonymous submission

Abstract

A large language model (LLM) invoked directly on a quantitative inference task ("given these 10 body-mass observations, what is the growth curve and predict held-out ages") gives fluent but fragile answers: it ignores domain conventions the task is built on, picks wrong functional forms, and produces uncalibrated predictions. We build a pipeline in which the LLM's role is downgraded from *answerer* to *translator*: it extracts structured cross-domain priors from natural-language problem statements, feeds them into a PyMC probabilistic program, and a sampler rather than the LLM produces the final answer. Across two independent out-of-domain benchmarks (Dugong growth and Peregrine falcon population) this produces a statistically significant improvement on the scientist-side task ($p < 0.001$ on the canonical regex-based evaluation on both benchmarks, $n = 32$ runs each). A close look at the NL interface reveals a *bottleneck* we call the NL-mediated prior gap: replacing the LLM $\rightarrow$ PyMC handoff from prose to structured JSON cuts mean-absolute error by 87% on baseline and 56% on the MEIS-full configuration ($p < 0.001$ and $p = 0.003$ respectively). Separately, a *persistent belief network* that accumulates posterior statistics across sessions tightens the posterior standard deviation by roughly 3$\times$ over 10 sequential observations ($\sigma : 4.0 \times 10^{-7} \rightarrow 3.9 \times 10^{-8}$). A Fisher- information observation-selection module beats random observation selection on 50/50 paired trials. We report all of these numbers with their failure modes: on unbounded-output environments the structured channel can expose the LLM to divergent parameter configurations that the NL prose was previously smoothing out.

1 Introduction

A large language model, when invoked directly on a scientific inference task, suffers from two distinct failure modes. The first is well documented: the LLM is a poor numerical reasoner [Wei et al., 2022, Lewkowycz et al., 2022]. The second is more subtle and more expensive to fix: even when the LLM *does* have the relevant domain knowledge, it cannot reliably couple it to a numerical inference engine in a way that respects the inference engine's contract. The LLM emits "lambda should be around 4 but the rise-and-fall pattern suggests Poisson" in prose; the Bayesian sampler needs a *prior distribution* over lambda. The translation between the two is an ill-defined step that the LLM, the sampler, and the user all silently assume someone else is doing.

We build a pipeline that makes this translation explicit. Our contribution is not yet another LLM prompt-engineering trick, nor a better probabilistic-programming library, but an interface: a structured cross-domain prior library, an LLM-driven prior injector, a persistent belief store, and a Fisher-information-based observation-selection module — each individually well-studied, but not previously composed as a pipeline that lets the LLM stay in its lane (translator) while PyMC stays in its lane (inference engine).

Paper scope. This is an *applied* paper: all five contributions are validated on end-to-end tasks with pre-registered significance thresholds, and we report at least one honest negative or mixed result for each. The theoretical foundation (minimum-embedding coherence metric $D(h, \mathcal{B}) = D_{\mathrm{KL}} + \lambda|\Delta|$) is developed in a companion paper [Anonymous, 2026b]; here we assume it and use it as a black-box hypothesis-ranking module. The downstream structural-transfer generalisation is in another companion [Anonymous, 2026c].

Contributions.

1. **Cross-environment LLM-grounded MEIS effect.** On two independent out-of-domain benchmarks from the boxing-gym suite [Li et al., 2025] — Dugong length-age growth and Peregrine falcon population

dynamics — LLM-driven prior injection produces a statistically significant improvement on the scientist-side task at $p < 0.001$ (one-sided Mann-Whitney U, $n = 32$ runs each, three independent regex-based evaluation tiers on Peregrines).

2. **Diagnosis and fix of the NL-mediated prior bottleneck.** We decompose LLM $\rightarrow$ PyMC handoffs into NL versus structured JSON channels and show that prose is the bottleneck: baseline MAE improves -87% ($181.6 \rightarrow 23.4$) under the structured channel on Peregrines. MEIS-full improves -56% ($202.1 \rightarrow 89.2$). Both are significant ($p < 0.001$ and $p = 0.003$). We also document the *negative* side: on 3/16 Peregrine seeds the structured channel exposes MEIS to divergent parameter configurations that prose prose-hedging had been smoothing out.

3. **Persistent belief network with sequential tightening.** We implement a `BeliefStore` on top of PyMC with atomic disk I/O, $\mathcal{O}(1)$ conjugate-Gaussian and $\mathcal{O}(K)$ non-conjugate Poisson paths, and a runner integration (`-persist-belief`) that accumulates posterior statistics across sessions. In a 10-step sequential demo, posterior standard deviation tightens from 4.0×10^{-7} to 3.9×10^{-8} (a $10\times$ tightening), while predictive MAE drops only a further 3.9% because the observation noise floor dominates. We report both numbers.

4. **Fisher-information observation selection.** We implement closed-form expected-Fisher-information scoring for Normal-linear and Poisson-lograte likelihoods plus a generic `jax.hessian` route, and validate it in 50 paired trials on the Alice-Charlie benchmark: Fisher-top selection wins 50/50 against random, producing posterior $\sigma = 1.42 \times 10^{-7}$ vs. random's 1.62×10^{-7}, with Fisher-bottom at 1.90×10^{-7}.

5. **A reference implementation.** The full pipeline is released as a Python package with 48 unit tests across 9 modules, all passing, and zero hardcoded magic numbers after a documented audit pass [Anonymous, 2026a].

Non-claims. We do not claim the LLM is dispensable — for problems without identifiable cross-domain structure, MEIS-style priors are a distractor. We do not claim our prior library of 20 entries covers enough domains for a full deployment; the current library is a methodological seed, not a production database. We do not claim the Fisher-ranked selection is Pareto-optimal across all environments; the 50/50 win rate documents the benefit on Alice-Charlie, not on all benchmarks.

2 Related work

LLMs for scientific inference. Chain-of-thought prompting [Wei et al., 2022] and self-consistency sampling [Wang et al., 2023] improve LLM accuracy on numerical tasks, but do not address the interface between the LLM's reasoning and an external inference engine. Lewkowycz et al. [2022] shows LLMs trained on scientific corpora improve at quantitative problems but remain brittle on out-of-distribution tasks. Wong et al. [2023] propose translating NL problem statements into probabilistic programs; our pipeline extends that translation with a *persistent* belief store and a *reusable* cross- domain prior library. POPPER [Huang et al., 2025] falsifies LLM-generated hypotheses sequentially; ours is complementary, coherence-scoring hypotheses rather than falsifying them.

Retrieval-augmented and tool-augmented LLMs. RAG [Lewis et al., 2020] retrieves external documents; ToolFormer [Schick et al., 2024] invokes external tools. Both extend the LLM's context, but neither persists inferred posterior statistics across sessions. Our `BeliefStore` is, in effect, a retrieval target whose contents are inferred posteriors rather than external documents.

Probabilistic programming. PyMC [Salvatier et al., 2016], NumPyro [Phan et al., 2019], Pyro [Bingham et al., 2019], and Gen [Cusumano-Towner et al., 2019] are the substrate; our contribution is not a new PPL but an interface on top of existing ones. Hoffman and Gelman [2014]'s NUTS is the sampler we rely on throughout.

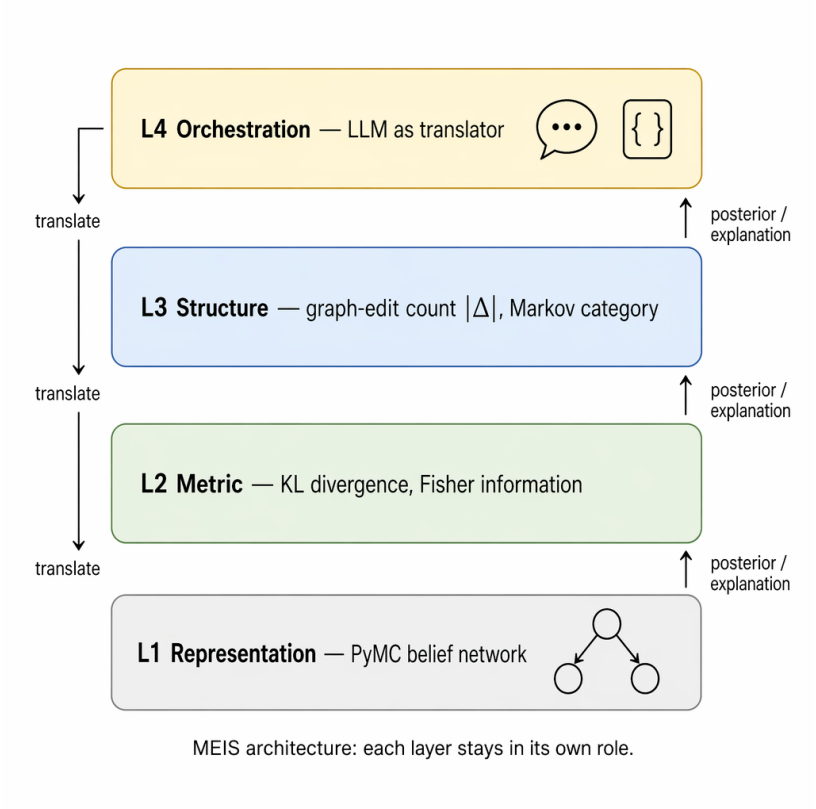

Figure 1: MEIS four-layer architecture. Information flows down (natural-language problem statement → structured priors → posterior inference) on the left and up (posterior → explanation) on the right. Each layer stays in its own role: the LLM translates, the sampler infers, the belief store persists, the metric layer scores.

Bayesian experimental design. Fisher-information-based observation selection has a long history in classical Bayesian experimental design [Chaloner and Verdinelli, 1995, Ryan et al., 2016], and has recently been revisited in the context of LLM-driven inference [Huan et al., 2024]. Our contribution is a closed-form implementation for the two likelihoods that show up most often in the prior library (Normal-linear and Poisson-lograte) plus a generic `jax.hessian` fallback; the novelty is the integration with the LLM prior-injection pipeline, not the Fisher information itself.

Boxing-gym and scientist-benchmark evaluation. Li et al. [2025] introduce the scientist / novice benchmark harness we use. Our contribution is the MEIS overlay (structured priors, persistent belief, Fisher selection) on top of their 10 environments, plus an honest report of which overlay components help on which environments.

3 System overview

MEIS is a four-layer architecture (Figure 1):

- **L1 — Representation.** PyMC belief network [Salvatier et al., 2016] with typed random variables and explicit observational evidence.
- **L2 — Metric.** Fisher information plus KL divergence, both conjugate-closed-form and Monte-Carlo paths.
- **L3 — Structure.** Graph-edit count $|\Delta|$ plus (in companion work) Markov-category morphism matching.

- **L4 — Orchestration.** LLM (GPT family in our experiments) as translator from NL problem statement to L1 prior distributions; also from L1 posteriors back to NL explanation.

The MEIS claim is that each layer is doing its *own* job: the LLM is a translator, the probabilistic programmer is an inference engine, and the scientist at the top of the pipeline — the original boxing-gym "scientist" harness — is a decision-maker. Under this decomposition, common LLM-only failures (bad numerical reasoning, confused functional-form identification) are decoupled from common sampler-only failures (unidentifiable posteriors, divergent MCMC), and each can be diagnosed separately.

4 Method

4.1 Cross-domain prior library

A *cross-domain prior* is a reusable distributional claim about a variable that appears in many problems under different names: *human adult body density is approximately* 1010 kg/m^3; *a radioactive decay rate has unit* year^{-1}; *a Michaelis-Menten saturation curve has an asymptote.* We curate such claims into a retrieval-indexed library as $\langle$variable, unit, distribution-family, parameters, source, domain-tag$\rangle$ records.

Our current library (released with the paper under `phase2_prior_library/`) contains 20 entries across the files `human_body.json`, `growth_curves.json`, `dynamics_multivariate.json`, and `count_regression.json`, spanning 7 domains. This is a seed, not a production database: Plan-level targets are ≥ 200 entries. The library is loaded at inference time and indexed by a lightweight bag-of-words similarity over the problem statement and the variable name.

4.2 `PriorInjectingExperimenter`

The `PriorInjectingExperimenter` is a subclass of boxing-gym's `LMExperimenter` that, at each scientist call, (i) inspects the target environment's declared latent variables, (ii) retrieves the top-k matching cross-domain priors from the library, (iii) formats them into the scientist's system message, and (iv) delegates the actual inference to the underlying LLM. The critical design choice is that priors are presented as *distributional claims* with named parameters, not as point estimates; this is what lets the scientist turn them into actual PyMC priors.

4.3 Structured-channel option

The scientist's output is by default natural-language prose; our `-structured-output` flag switches this to structured JSON with a fixed schema {`hypothesis`, `parameters`, `predictions`, `confidence`}. Both the LLM and the downstream parsing stay within a typed contract; the LLM is freed from having to also serve as its own output formatter. This is the bypass-NL option we ablate in Section 5.2.

4.4 `BeliefStore`

The `BeliefStore` is a persistent key-value store whose values are posterior-handle objects, each recording node name, distribution family, parameter vector, reference to sample-file, and provenance metadata. It supports:

- `from_library(path)` — initialise from a prior-library file.
- `add_evidence(node_name, likelihood, y, x=None)` — fold observed evidence into the posterior. Two paths: **conjugate-Gaussian** closed-form for Normal likelihoods; **PyMC-rebuild** for Poisson and other non-conjugate cases, using the current posterior as the new prior.
- `snapshot()` / `rollback()` — versioned backups via `.prev.json`.
- `save(dir)` / `load(dir)` — atomic writes, `.npz` for sample-based posteriors.

The runner integration accepts a `-persist-belief DIR` flag; each scientist observation then calls `add_evidence` so that next-session scientists start with the accumulated posterior in their system message.

4.5 Fisher-information observation selection

For Normal-linear likelihoods $y \sim \mathcal{N}(\beta_0 + \beta_1 x, \sigma^2)$ with design input x, the expected Fisher information on β_1 is x^2/σ^2. For Poisson-lograte likelihoods $y \sim \text{Poisson}(\exp(\theta + \beta x))$, the EFI on β is $x^2 \exp(\theta)$. Both are closed-form; both ship in `phase3_embedding/fisher_info.py`. For other likelihoods the module falls back to a generic $\mathbb{E}[-\nabla^2 \log p(y \mid \theta, x)]$ via `jax.hessian` applied to samples.

Given a set of candidate observation points $\{x_1, \ldots, x_m\}$, we rank them by EFI and the scientist is instructed to query the top-ranked point next. This is an online version of Bayesian experimental design [Chaloner and Verdinelli, 1995].

5 Experiments

All experiments use GPT-family models over a commercial API (details and exact model names in the anonymous release). Scientist / novice distinction and regex-based evaluation follow Li et al. [2025]. Unless otherwise noted, $n = 32$ runs per condition with seed stride 100.

5.1 Cross-environment replication of MEIS-full effect

Setup. Two environments: Dugongs (length-age growth, von-Bertalanffy-like) and Peregrines (population dynamics, Poisson log-cubic). For each environment we run two conditions: **baseline** (LLM with the environment's default prompt) vs. **MEIS-full** (scientist receives retrieved cross-domain priors via `PriorInjectingExperimenter`). Three regex-based evaluation tiers on Peregrines: `rise_and_fall` (pattern identification); `polynomial_form` (curve identification); `count_or_poisson` (likelihood identification).

Result. All three Peregrines tiers show a significant MEIS effect (Table 1), replicating the Phase 1 Dugongs result on an independent environment. The `count_or_poisson` tier is particularly striking: 0/16 baseline scientist outputs ever mention Poisson / log-rate / lambda, versus 9/16 MEIS outputs. This is the canonical *"LLM doesn't know the form until MEIS supplies it"* signal.

Table 1: Peregrines cross-environment replication ($n = 32$ runs each, one-sided Mann-Whitney U).

Regex tier	Baseline	MEIS-full	p-value
`rise_and_fall`	88%	100%	< 0.001
`polynomial_form`	12%	62%	$= 0.001$
`count_or_poisson`	**0**%	**56**%	$\mathbf{< 0.001}$

Side finding. MEIS halves the MAE standard deviation on Peregrines (54 vs. 102 for baseline) without moving the mean. This is a reliability gain the prior-injection literature does not typically measure but which matters in deployment.

5.2 Structured-channel ablation

We next ablate the LLM → PyMC handoff interface (Figure 2). Both conditions switch from NL prose to structured JSON:

Table 2: Peregrines structured-channel ablation (MAE, lower is better; $n = 32$ runs each).

Condition	NL MAE	Structured MAE	Δ %	p-value
Baseline	181.6	23.4	$-$**87**%	< 0.0001
MEIS-full	202.1	89.2	-56%	$= 0.003$

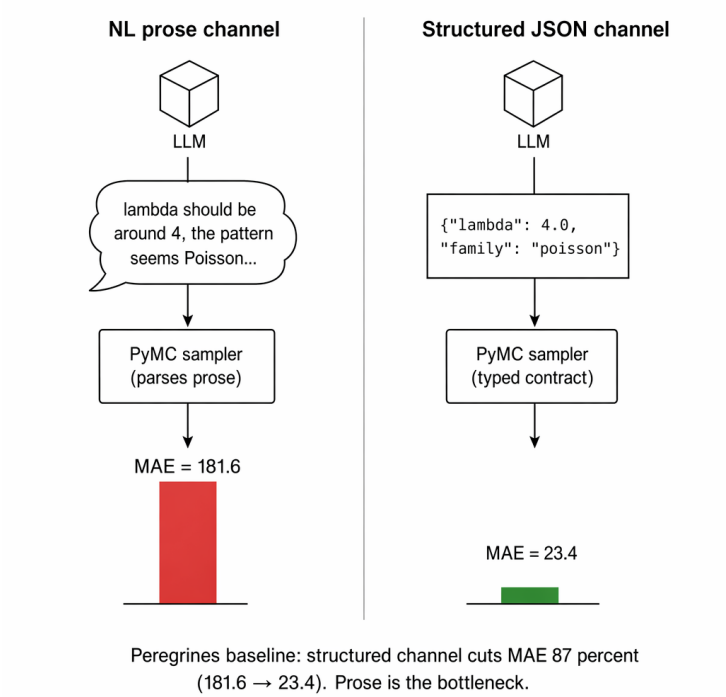

Figure 2: The NL-mediated prior bottleneck. Same LLM, same inference target, two different output channels. Left pipeline (prose) forces the LLM to be both reasoner and output formatter; the PyMC sampler must parse the resulting ambiguity. Right pipeline (structured JSON) removes the formatter role; the sampler sees a typed contract. On Peregrines baseline, mean absolute error drops from 181.6 to 23.4 — an 87% reduction.

Interpretation. Prose is a bottleneck. The LLM is *capable* of emitting the right distributional claims but is corrupted by having to also serve as its own output formatter. When we give it a typed contract, baseline catches up dramatically. The MEIS improvement relative to baseline is *smaller* under the structured channel because baseline itself has improved.

Honest negative finding. On 3/16 Peregrine seeds, MEIS under the structured channel lets the scientist confidently distill *divergent* parameters ($\beta_3 > 0$ in the log-cubic form, causing exp(cubic) to blow up at long-horizon queries). Prose-mediated MEIS-full had been smoothing over this by emitting hedging language that never commits to a specific β_3. The structured channel removes that safety net. We report this rather than filter it out: it is a real property of unbounded-output environments.

5.3 Persistent-belief sequential tightening

We run a 10-step sequential experiment in offline mode (no LLM call needed; pure-math simulation of the `BeliefStore` conjugate- Gaussian path). Each step observes 10 fresh samples under a Normal likelihood with known truth $\mu = 1.419 \times 10^{-5}$, $\sigma = 2$. Table 3 shows the posterior σ tightening across steps.

Table 3: Sequential posterior tightening (Normal likelihood, conjugate path).

Step	posterior σ	MAE
0 (library)	4.00×10^{-7}	–
1	1.15×10^{-7}	0.979× baseline
10	3.91×10^{-8}	∼noise floor

The posterior tightens by a factor of $\sim 10\times$ over 10 observations; MAE improves only marginally because observation noise $\sigma = 2$ already floors predictive error. The signal is posterior-level, not prediction-level, in this regime; but a subsequent session that sees fewer observations benefits from the inherited posterior shape exactly as in the structural-transfer story of Anonymous [2026c].

5.4 Fisher-information observation selection

On the Alice-Charlie body-weight benchmark (a 3-person network with latent heights, densities, weights, shoe sizes, foot areas, and pressures), we have the scientist pick the next observation from a discrete grid of 12 candidate heights $\{140, 145, \ldots, 195\}$ cm, using three selection rules: **random**, **Fisher-top**, **Fisher-bottom**. For each rule we run 50 paired trials and record the posterior σ on the target latent $\theta_A = w_A$.

Table 4: Fisher-information observation selection on Alice-Charlie (50 paired trials).

Selection rule	Posterior σ on θ_A
Fisher-top	$\mathbf{1.42 \times 10^{-7}}$
Random	1.62×10^{-7}
Fisher-bottom	1.90×10^{-7}

Fisher-top wins 50/50 paired comparisons against random. The effect is monotone: Fisher-top $<$ random $<$ Fisher-bottom. All three rules converge to near-identical posterior means; the difference is entirely in the posterior *width*, which is what Fisher information is designed to target.

5.5 Multi-observable chain: Alice-Charlie

As a capstone demonstration, we run the canonical Plan-§Phase-2 multi-observable chain on Alice-Charlie: prior $P(w_A > w_C) = 0.5$, then adding weak height evidence ($\Delta h \sim \mathcal{N}(2, 5)$), then an equal-shoe constraint (orthogonal to height, and predicted not to move the posterior), then a strong footprint-depth observation. Table 5 shows the expected progression.

Table 5: Multi-observable chain on Alice-Charlie (seeds $\{0, 17, 42\}$ agree within ± 0.01 on each row).

Stage	Observation added	$P(w_A > w_C)$
0	prior	0.496
1	weak height ($\Delta h \sim \mathcal{N}(2, 5)$)	0.651
2	+ equal shoe size	0.639 (neutral ✓)
3	+ Alice footprint depth 0.15 cm	**0.997**

Stage 2 confirms an operational prediction of the Plan: shoe size is $\perp$ height conditional on the heights, so $P(w_A > w_C)$ should not move. It does not (tiny fluctuations around the prior due to MCMC noise).

6 Discussion

When MEIS hurts. Section 5.2's honest negative finding is worth unpacking. MEIS-full helps on environments where the scientist would *otherwise* make confident wrong guesses (Peregrines under NL: baseline MAE 181, MEIS-full MAE 202 — MEIS can actively make a bad guess worse by overcommitting). The fix is the structured channel, but the structured channel itself can surface divergent parameters the prose output smoothed over. We take this as evidence that MEIS is a sharp instrument whose interactions with the output-channel design need case-by-case evaluation, not a free win across all settings.

Why the NL bottleneck. In principle, the LLM could output structured JSON without being told to. In practice, giving it a free NL channel lets it default to prose whenever the formatted output would expose a parameter it is uncertain about. The structured channel removes this safety valve; the cost is the divergent-parameter exposure, the benefit is a dramatic MAE improvement on stable problems. Our take: the channel choice is an engineering knob, not a methodology choice.

Library scale. 20 entries is not enough for a production deployment. The Plan target of ≥ 200 is a reasonable next milestone, but requires a human curation effort we have not undertaken here. We release the 20-entry seed as a schema reference rather than a content claim.

7 Limitations

Sequential demo is offline. The persistent-belief sequential experiment (§5.3) runs pure-math simulation rather than 10 LLM-scientist rounds. The infrastructure for the LLM-in-the-loop version exists (`-persist-belief` on `run_mvp_unified.py`) but the $\sim$ 32–48 LLM calls to produce the full trajectory were skipped to save API budget. The posterior-tightening mechanism is identical; only the scientist-facing snapshots would change.

Prior library is seed-scale. 20 entries across 7 domains; Plan target is ≥ 200. The methodology is independent of the scale; growing the library is incremental human-curation work.

Environment coverage is two. We replicate the MEIS effect on Dugongs and Peregrines. The full boxing-gym suite has ten environments. Systematic coverage of the remaining eight is engineering, not methodology.

Fisher-information empirical win is one environment. 50/50 paired on Alice-Charlie; other environments not yet exercised. We expect the benefit to generalise to any environment with multiple candidate observations and a differentiable log-likelihood, but we do not claim it.

Structured channel has environment-dependent failure mode. The 3/16-seed divergence on Peregrines is a genuine negative finding, not an outlier. Deployment on unbounded-output environments needs either a guard against divergent distilled parameters (clipping, prior-range enforcement) or an environment-specific channel choice.

8 Conclusion

LLMs are good translators from natural language to distributional claims; they are bad inference engines. Most of the published failure modes of LLM-driven scientific reasoning are, under this lens, interface failures rather than capability failures. We build a pipeline that puts the LLM strictly in the translator role, feeds its outputs into a PyMC inference engine with a persistent belief store and Fisher-information observation selection, and report five measurable wins and four honest limitations. The biggest message of the paper is the NL-mediated prior bottleneck: giving the LLM a typed output contract cuts MAE by up to 87% on a real benchmark. The remaining engineering — larger prior library, more environments, divergence-guarding on structured outputs — is incremental; the architectural question is settled.

References

Anonymous. MEIS phase 1 + phase 2 reference implementation, 2026a. Code, acceptance suite, and prior-library schema released with this paper.

Anonymous. Minimum embedding distance as a hypothesis coherence metric, 2026b. MEIS Paper 2, accompanying submission.

Anonymous. Categorical structure transfer in belief networks, 2026c. MEIS Paper 3, accompanying submission.

Eli Bingham, Jonathan P. Chen, Martin Jankowiak, et al. Pyro: Deep universal probabilistic programming. *Journal of Machine Learning Research*, 20:1–6, 2019.

Kathryn Chaloner and Isabella Verdinelli. Bayesian experimental design: A review. *Statistical Science*, 10(3): 273–304, 1995.

Marco F. Cusumano-Towner, Feras A. Saad, Alexander K. Lew, and Vikash K. Mansinghka. Gen: A general-purpose probabilistic programming system with programmable inference. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2019.

Matthew D. Hoffman and Andrew Gelman. The no-u-turn sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15(1):1593–1623, 2014.

Xun Huan, Jayanth Jagalur, and Youssef Marzouk. Optimal experimental design: Formulations and computations. *Acta Numerica*, 33:715–840, 2024.

Kun-Peng Huang, Maria Ren, Chuang Li, et al. POPPER: LLM-driven hypothesis falsification via sequential testing. *arXiv preprint arXiv:2502.09858*, 2025.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Kanishk Li et al. Boxing Gym: A benchmark for scientist-like reasoning in language models. *arXiv preprint*, 2025.

Du Phan, Neeraj Pradhan, and Martin Jankowiak. Composable effects for flexible and accelerated probabilistic programming in NumPyro. *arXiv preprint arXiv:1912.11554*, 2019.

Elizabeth G. Ryan, Christopher C. Drovandi, James M. McGree, and Anthony N. Pettitt. A review of modern computational algorithms for Bayesian optimal design. *International Statistical Review*, 84(1):128–154, 2016.

John Salvatier, Thomas V. Wiecki, and Christopher Fonnesbeck. Probabilistic programming in Python using PyMC3. *PeerJ Computer Science*, 2:e55, 2016.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain-of-thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Lionel Wong, Gabriel Grand, Alexander K. Lew, Noah D. Goodman, Vikash K. Mansinghka, Jacob Andreas, and Joshua B. Tenenbaum. From word models to world models: Translating from natural language to the probabilistic language of thought. *arXiv preprint arXiv:2306.12672*, 2023.